

Student Level Conditioned StepDPO for Math Tutor LLMs

ChaeWon An^{1†}, Minwoo Park^{1†}, Yejin Yoon^{2†}, Jihye Kim^{2†}, Jong-June Jeon^{1*}

¹ Department of Statistics, University of Seoul, S. Korea

² Department of Statistics and Data Science, University of Seoul, S. Korea



Contribution

We propose **SLC-StepDPO**, a student level conditioned StepDPO method for math tutor LLMs to provide prefer solution steps that are both mathematically correct and appropriate for the target student level.

1. We define a student level condition c from the Mathematics Curriculum announced by the Ministry of Education of the Republic of Korea.
2. We construct **conditioned win-lose step comparisons** represented as $(x, c, s_{1:k-1}, s_w, s_l)$. The losing step s_l can be rejected because it contains a math error or because its explanation level does not match the target condition c .
3. We train LLM using a unified StepDPO loss after SFT, so that the preferred step changes according to both mathematical validity and student level appropriateness.

Introduction

Motivation Educational LLMs used as AI tutors should satisfy two requirements. They should generate correct solution steps, and they should explain those steps at a level the student can understand. A tutoring response is not sufficient when it gives a correct answer with language that is too advanced, or when it uses accessible language but contains an error in the solution process.

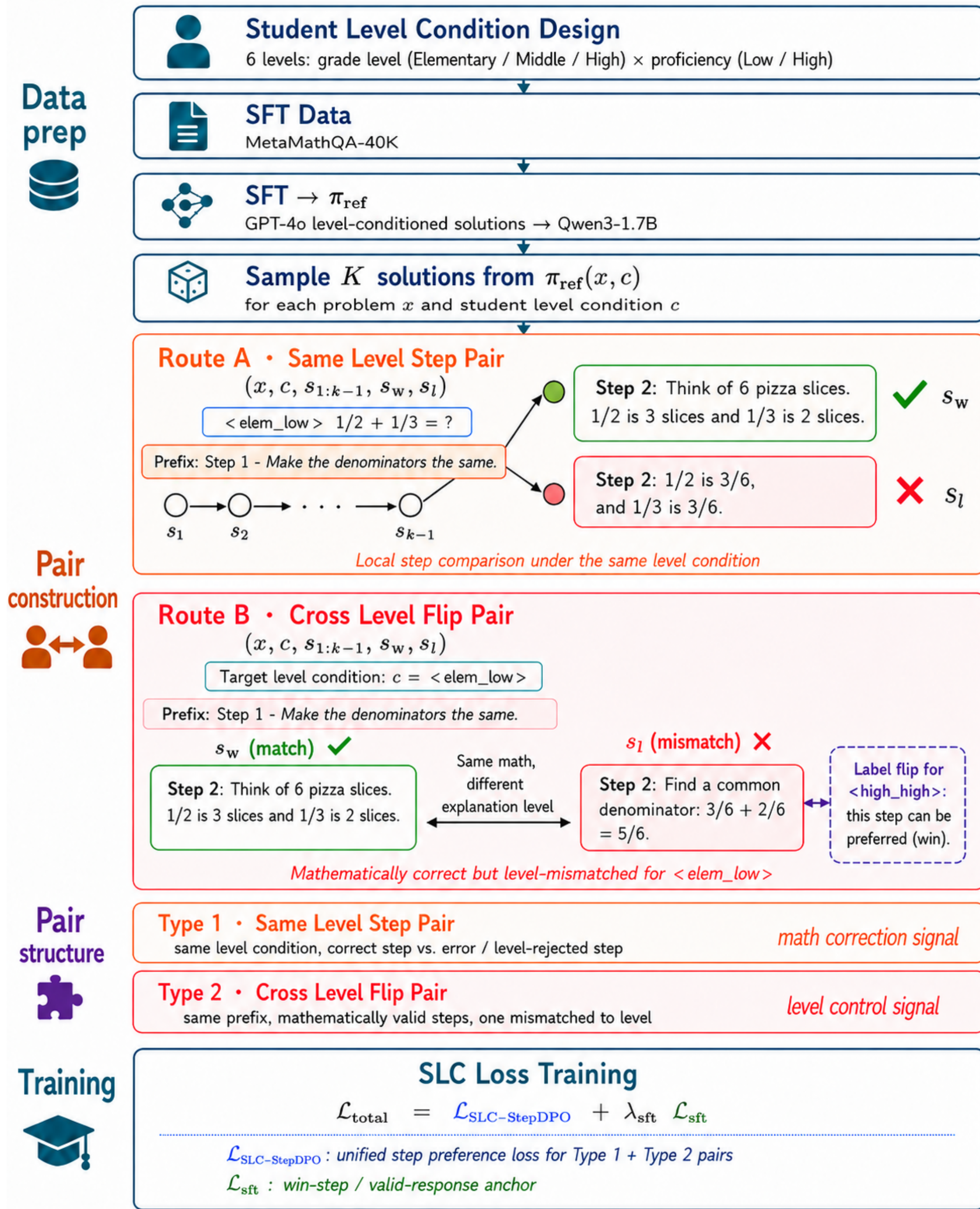
Prior Work StepDPO improves mathematical reasoning by comparing the first incorrect step with a corrected step under the same prefix. This provides a local learning signal for step-level error correction. GDPO introduces a student-level condition variable for group-conditioned preference learning, allowing the model to represent different preference patterns across learner groups.

Notations

x : math problem text
 a^* : reference final answer for x
 $c \in \mathcal{C}$: student level condition
 κ_c : curriculum constraint profile for c
 π_{ref} : fixed SFT reference model
 π_θ : policy model being optimized

$y = (s_1, \dots, s_K)$: step wise solution
 s_i : the i th reasoning step
 $s_{1:k-1}$: prefix before step k
 s_w, s_l : winning and losing steps
 $\mathcal{D}_{\text{step}}$: same level step pairs
 $\mathcal{D}_{\text{flip}}$: cross level flip pairs

Methods



Data generation for Supervised Fine Tuning We use MetaMathQA benchmark dataset for extracting the problem text x , the reference final answer a^* . We also define student-level condition $c \in \mathcal{C}$ from two axes: grade level (Elementary, Middle School, High School) and proficiency (Low, High). Each student-level condition c is represented by a tag such as **<elem_low>** or **<high_high>**. Curriculum constraint profile κ_c is derived from the 2022 Revised National Curriculum, Mathematics Curriculum, Annex 8, announced by the Ministry of Education of the Republic of Korea (Ministry of Education Notice No. 2022-33). For each problem x and student-level condition c , GPT-4o generates a step by step tutor solution under the constraint profile κ_c . Each generated response follows a fixed format. The SFT model is fine tuned on these generated student-level conditioned solutions.

Preference Data Construction Preference pairs are constructed in two stages, but both stages are canonicalized into the same step pair schema: $(x, c, s_{1:k-1}, s_w, s_l)$.

Stage A: Same Level Step Pair

- Sample step wise solutions from $\pi_{\text{ref}}(\cdot | x, c)$.
- Label each candidate step as **acceptable**, **reject_math**, or **reject_level**.
- Group candidate steps by the same problem, level condition, and prefix $(x, c, s_{1:k-1})$.
- Pair an acceptable step with a rejected step: $(x, c, s_{1:k-1}, s_w, s_l) \in \mathcal{D}_{\text{step}}$.

Here, s_l can be a math error or a level mismatched step under the same student level condition.

① Shared Step Preference Score All training pairs have the same canonical schema $(x, c, s_{1:k-1}, s_w, s_l)$. Given a problem x , student level condition c , prefix $s_{1:k-1}$, winning step s_w , and losing step s_l , we define

$$\Delta_\theta(x, c, s_{1:k-1}, s_w, s_l) = [\log \pi_\theta(s_w | x, c, s_{1:k-1}) - \log \pi_{\text{ref}}(s_w | x, c, s_{1:k-1})] - [\log \pi_\theta(s_l | x, c, s_{1:k-1}) - \log \pi_{\text{ref}}(s_l | x, c, s_{1:k-1})].$$

② Unified Step Preference Loss $\mathcal{D}_{\text{train}}$ are trained with the step preference loss:

$$\mathcal{L}_{\text{SLC-StepDPO}}(\theta) = -\mathbb{E}_{(x, c, s_{1:k-1}, s_w, s_l) \sim \mathcal{D}_{\text{train}}} [\log \sigma(\beta \Delta_\theta(x, c, s_{1:k-1}, s_w, s_l))].$$

Type 1 pairs correct rejected steps under the same student level condition. Type 2 pairs make the same step change its preference label across different student level conditions.

③ SFT Anchor The SFT anchor keeps probability mass on the winning step:

$$\mathcal{L}_{\text{sft}}(\theta) = -\mathbb{E}_{(x, c, s_{1:k-1}, s_w) \sim \mathcal{D}_{\text{train}}} [\log \pi_\theta(s_w | x, c, s_{1:k-1})].$$

④ Total Loss

$$\mathcal{L}_{\text{Total}}(\theta) = \underbrace{\mathcal{L}_{\text{SLC-StepDPO}}(\theta)}_{(2) \text{ Type 1 + Type 2 preference loss}} + \lambda_{\text{sft}} \underbrace{\mathcal{L}_{\text{sft}}(\theta)}_{(3) \text{ win step anchor}}.$$

Role of each term. $\mathcal{L}_{\text{SLC-StepDPO}}$ scores both Type 1 and Type 2 pairs with the same Δ_θ . $\mathcal{L}_{\text{sft}}$ anchors the winning step.

Experiment

Setup

- Base model: Qwen3-1.7B with LoRA for parameter efficiency.
- Training data: Sampling 1,500 math problems from MetaMathQA-40K.
- Data generation : GPT-4o-mini for preference data generation.

Evaluation Metrics For each test problem x_i , the model generates a step-wise solution $y_i = (s_{i,1}, \dots, s_{i,K_i})$.

- FinalAcc = $\frac{1}{N} \sum_{i=1}^N 1[\hat{a}_i = a_i^*]$, StepAcc = $\frac{\sum_{i=1}^N \sum_{j=1}^{K_i} 1[s_{i,j} \text{ is mathematically valid}]}{\sum_{i=1}^N K_i}$
- Student level evaluation: ExpMatch = $\frac{\sum_{i=1}^N \sum_{j=1}^{K_i} 1[s_{i,j} \text{ matches student level condition } c_i]}{\sum_{i=1}^N K_i}$.
- Flip handling evaluation : For each problem, the model generates y_i^{lo} under c_{lo} and y_i^{hi} under c_{hi} . FlipHandling = $\frac{1}{N} \sum_{i=1}^N M(y_i^{lo}, c_{lo}) \cdot M(y_i^{hi}, c_{hi}) \cdot (1 - M(y_i^{hi}, c_{lo}))$

Model	Final Acc.↑	Step Acc.↑	Explanation Match↑	Belief Flip
SFT	73.9	91.5	79.5	8.3
StepDPO	72.2	90.9	79.1	10.0
SLC-StepDPO($\lambda_{\text{sft}} = 0.01$)	73.3	92.0	80.9	10.0

Table 1. Held-out evaluation with no train/eval overlap: 360 solutions (60 problems $\times$ 6 student levels), scored by an independent GPT-4o judge.

Example. Both models reach 13/20, but SLC-StepDPO gives a level-appropriate explanation.

Problem: Yuna ate 1/4 of a pizza and her brother ate 2/5 of it. How much did they eat together? Student level condition: Elementary, grade 3-4 (age 8-10)	<table> <tr> <th>Step-DPO (baseline)</th><th>SLC-StepDPO (Ours)</th></tr> <tr> <td> Step 1: First, find how much pizza Yuna ate. She ate 1/4 of the pizza. Step 2: Next, her brother ate 2/5 of the pizza. Step 3: To find the total, we add the fractions 1/4 and 2/5. Step 4: To add these fractions, we need a common denominator. The least common denominator for 4 and 5 is 20. Step 5: Convert 1/4 to 5/20 and 2/5 to 8/20. Add them to get 13/20. Final answer: 13/20 <i>Uses a formal term without concrete grounding for an elementary learner.</i> </td><td> Step 1: First, find how much Yuna ate. Think of the pizza as 4 equal parts. Yuna took 1 part, so she ate 1 out of 4 parts. Step 2: Now her brother. Imagine the pizza as 5 equal parts. He took 2 parts, so 2 out of 5 parts. Step 3: To add them, think of the pizza as having 20 equal parts. Yuna ate 5 parts and her brother ate 8 parts, so 5 + 8 = 13 parts. Step 4: The whole pizza is 20 parts, so together they ate 13 out of 20. Final answer: 13/20 <i>Grounds each step in equal parts, which is more appropriate for this student level.</i> </td></tr> </table>	Step-DPO (baseline)	SLC-StepDPO (Ours)	Step 1: First, find how much pizza Yuna ate. She ate 1/4 of the pizza. Step 2: Next, her brother ate 2/5 of the pizza. Step 3: To find the total, we add the fractions 1/4 and 2/5. Step 4: To add these fractions, we need a common denominator . The least common denominator for 4 and 5 is 20. Step 5: Convert 1/4 to 5/20 and 2/5 to 8/20. Add them to get 13/20. Final answer: 13/20 <i>Uses a formal term without concrete grounding for an elementary learner.</i>	Step 1: First, find how much Yuna ate. Think of the pizza as 4 equal parts . Yuna took 1 part, so she ate 1 out of 4 parts. Step 2: Now her brother. Imagine the pizza as 5 equal parts . He took 2 parts, so 2 out of 5 parts. Step 3: To add them, think of the pizza as having 20 equal parts . Yuna ate 5 parts and her brother ate 8 parts, so 5 + 8 = 13 parts. Step 4: The whole pizza is 20 parts, so together they ate 13 out of 20. Final answer: 13/20 <i>Grounds each step in equal parts, which is more appropriate for this student level.</i>
Step-DPO (baseline)	SLC-StepDPO (Ours)				
Step 1: First, find how much pizza Yuna ate. She ate 1/4 of the pizza. Step 2: Next, her brother ate 2/5 of the pizza. Step 3: To find the total, we add the fractions 1/4 and 2/5. Step 4: To add these fractions, we need a common denominator . The least common denominator for 4 and 5 is 20. Step 5: Convert 1/4 to 5/20 and 2/5 to 8/20. Add them to get 13/20. Final answer: 13/20 <i>Uses a formal term without concrete grounding for an elementary learner.</i>	Step 1: First, find how much Yuna ate. Think of the pizza as 4 equal parts . Yuna took 1 part, so she ate 1 out of 4 parts. Step 2: Now her brother. Imagine the pizza as 5 equal parts . He took 2 parts, so 2 out of 5 parts. Step 3: To add them, think of the pizza as having 20 equal parts . Yuna ate 5 parts and her brother ate 8 parts, so 5 + 8 = 13 parts. Step 4: The whole pizza is 20 parts, so together they ate 13 out of 20. Final answer: 13/20 <i>Grounds each step in equal parts, which is more appropriate for this student level.</i>				

Acknowledgments

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Ministry of Science and ICT (MSIT) (No. RS-2022-NR068754).